Automation and Robotics: Methods and Systems of Production Control

Learning Outcomes

- Explain what operations research is and why it matters in production control
- Distinguish descriptive analytics, prediction, and optimization
- Formulate a simple linear programming model from a production problem
- Interpret a feasible region, an optimal solution, and the basic logic of the Simplex method

Lecture Route and Suggested Pauses

- **Core:** OR decisions, the Wyndor model, graphical solution, and resource interpretation.
- **Worked derivation:** follow the Simplex path first. The detailed tableau sequence can serve as a second pass or reading after class.
- **Transfer:** blend ingredients and route shipments while checking units and conservation.
- **Companion:** use the notebook after the graphical solution to test a capacity purchase before returning to algebra.

What This Course Covers

- Analysis and improvement of industrial and business processes
- Focus on maximizing results under limited resources
- Based on optimization, operations research, and machine learning
- Success depends on correctly defining the problem

Organizational Structure

- Final quiz includes questions on the use of optimization and machine learning in industry
- Labs are hands-on with optimization/ML problems
- Tools covered:
  - Python, PuLP, scikit-learn
  - Jupyter, Google Colab, Git
- Lectures focus on understanding the use of optimization and machine learning in industry and problem formulation

Origins of Operations Research

- WWII: logistics and prioritization (radar placement, convoys)
- Similar challenges in industry and Cold War
- Rise of computers and algorithms (e.g., 1947 Simplex method)

Operations Research

- Optimizing operations within organizations
- Scientifically grounded methods
- Steps:
  - Observe, define problem, gather data
  - Build, validate, deploy and maintain models

Basic Examples of Operations Research

- Production planning: what and when to produce
- Logistics: assigning trucks to routes
- Tram schedules, school timetables
- General focus on large-scale problems

Why This Matters in Automation and Robotics

- Production control decides what to produce, when, where, and with which resources
- OR supports scheduling, sequencing, inventory control, routing, and capacity allocation
- Machine learning supports forecasting, quality prediction, anomaly detection, and predictive maintenance
- Robotics systems benefit when better predictions and better optimization are combined in one decision pipeline

Analytics, Big Data, and ML

- Turning data into business intelligence
- Key questions:
  - Where are we? (Descriptive analytics)
  - Can we predict?
  - What should we do?

Examples of Data-Driven Problems

- Best-performing advertisements
- Product recommendations
- Predictive maintenance for machines
- Credit default prediction

Lecture Topics (Part 1)

- Introduction and motivation
- Review of basic production control methods
- Linear Programming:
  - Concepts, formulation, limitations
  - Simplex algorithm
- Nonlinear Optimization and Dynamic Programming:
  - Fundamental ideas and use cases

Lecture Topics (Part 2)

- Analysis and modeling of technological processes
- Project scheduling and network planning methods:
  - Critical Path Method (CPM)
  - PERT (Program Evaluation and Review Technique)
  - Gantt charts
- Machine Learning for decision support in production control:
  - Concepts, types of learning
  - Applications in industrial analytics and optimization

Operations Research Summary

- OR = optimizing institutional processes
- A toolbox of scientifically supported methods
- Applicable across domains

Tools in OR

- Algorithms: Linear and Integer Programming
- Solvers: Gurobi, CPLEX
- Libraries: PuLP, Pyomo, Google OR-Tools
- Specialized software
- Most important: the modeling process

Further Exploration

- Who is the operations research analyst?
  `https://www.youtube.com/watch?v=9z6jFPxcwz8`
- What is operations research:
  `https://www.youtube.com/watch?v=0oMVVx81kCs`
- Topics link to cybernetics, automation, and interdisciplinary fields

Phases of an Operations Research (OR) Project

The usual (overlapping) phases of an OR study are:

1. Define the problem and gather data
2. Formulate a mathematical model
3. Develop a solution procedure
4. Test and refine the model
5. Prepare for ongoing use
6. Implement the model

Phase 1: Define the Problem and Gather Data

- Study the system and create a clear problem statement
- Identify objectives, constraints, alternatives, and timelines
- Understand the decision-maker's goals
- Gather relevant data (may require data mining or new systems)

Phase 2: Formulate a Mathematical Model

- Define decision variables, objective function, and constraints
- Determine model parameters and handle uncertainty (e.g., sensitivity analysis)
- Start simple and refine as needed
- Balance between model accuracy and solvability

Phase 3: Develop a Solution Procedure

- Use algorithms or software tools to solve the model
- Understand that solutions are only as good as the model
- Use heuristics or metaheuristics when exact methods are impractical
- Postoptimality analysis may be needed

Phase 4: Test and Refine the Model

- Validate the model thoroughly (e.g., dimensional checks, historical data tests)
- Engage future users in testing
- Document the validation process
- Debug and improve as necessary

Phase 5: Prepare for Ongoing Use

- Design systems for repeatable application of the model
- Integrate with existing information systems (e.g., MIS, databases)
- Establish procedures for maintenance and future updates

Phase 6: Implement the Model

- Translate model outputs into real-world procedures
- Collaborate with management and train personnel
- Monitor system performance and adjust as needed
- Ensure ongoing support and involvement from leadership

An OR Project in a Robot Assembly Cell

Illustrative teaching scenario: kits queue before a shared tester.

- **Decision:** choose tomorrow's kit quantities, before the shift starts.
- **Data:** minutes per kit, available station minutes, contribution profit, orders.
- **Model:** maximize daily contribution subject to station capacity and orders.
- **Validation:** replay a past day. Do setup and rework consume time the model omitted?
- **Deployment:** the planner reviews a proposed mix and records actual station use.

Discussion: A more accurate demand forecast cannot create more tester time. Which part of this problem needs prediction, and which part needs optimization?

Predict-Then-Optimize Paradigm

- ML models are used to generate predictions or extract patterns from data
- These predictions serve as inputs to OR models for optimal decision-making

Key Advantages of OR + ML Integration

- Combines data-driven intelligence (ML) with decision-making rigor (OR)
- Enables more accurate, context-aware optimization
- Supports tackling complex, real-world problems in a scalable manner

How to optimize production? Case Study: Wyndor Glass Co.

- Wyndor Glass Co. produces high-quality glass doors and windows
- Facing declining earnings, management discontinued unprofitable products
- Two new high-potential products were launched:
  - Product 1: 8-foot glass door with aluminum framing
  - Product 2: 4×6 foot double-hung wood-framed window
- Goal: Determine optimal weekly production rates to maximize profit

Production Structure

Plant Utilization:

- **Plant 1:** Aluminum framing – used by Product 1
- **Plant 2:** Wood framing – used by Product 2
- **Plant 3:** Glass and assembly – used by both products

Business Insight:

- Marketing believes all produced units can be sold
- Key limitation: shared capacity in Plant 3

Problem Setup

Objective: Maximize total weekly profit from the two new products
Decision Variables:

- x_1: Batches of Product 1 per week
- x_2: Batches of Product 2 per week

Required Data:

- Available production hours per plant
- Hours required per batch for each product
- Profit per batch

Production Data Summary (Table 3.1)

Plant	Available Hours	Hours per Batch (P1)	Hours per Batch (P2)
1	4	1	0
2	12	0	2
3	18	3	2

Profit per Batch:

- Product 1: \$3,000
- Product 2: \$5,000

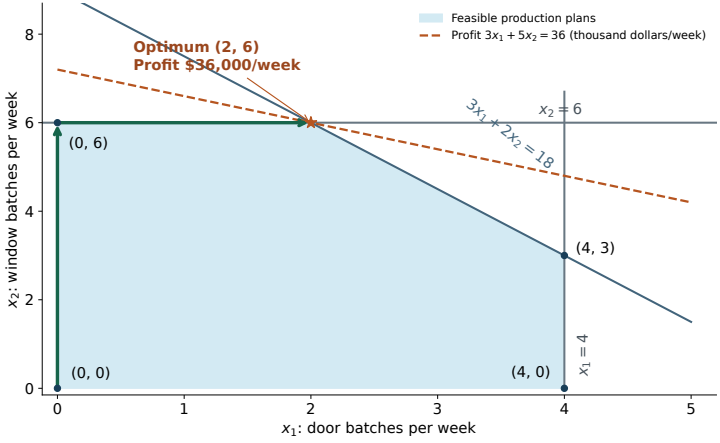
Maximize weekly profit (in thousands of dollars):

$$Z = 3x_1 + 5x_2$$

Subject to:

$$\begin{aligned} x_1 &\leq 4 && \text{(Plant 1)} \\ 2x_2 &\leq 12 && \text{(Plant 2)} \\ 3x_1 + 2x_2 &\leq 18 && \text{(Plant 3)} \\ x_1, x_2 &\geq 0 && \text{(Non-negativity)} \end{aligned}$$

Graphical Solution of Wyndor Glass Co. Problem



Blue shading contains feasible plans. Orange marks the optimum $(2, 6)$, with \$36,000/week contribution. Green shows one Simplex path.

Graphical Solution Summary

- The problem involves two decision variables $\rightarrow$ solvable graphically
- Feasible region defined by intersection of all constraints
- Iso-profit lines used to find the point farthest in the optimal direction
- **Optimal solution:** $x_1 = 2$, $x_2 = 6$
- **Maximum profit:** \$36,000 per week

The Production Plan as an Operational Decision

At $(x_1, x_2) = (2, 6)$, weekly use and unused capacity are:

Plant	Used hours	Unused hours
1	2	$4 - 2 = 2$
2	$2(6) = 12$	0
3	$3(2) + 2(6) = 18$	0

- Producing only the higher-profit product gives $(0, 6)$ and $Z = 30$.
- Two batches of Product 1 use the remaining six Plant 3 hours and add 6 to profit.
- **Predict:** Would one more hour in Plant 1 help? What about Plant 3?

Beyond Graphical Solutions: The Need for Simplex

- Graphical method works well for 2 variables (like Wyndor Glass).
- What about problems with 3, 10, or thousands of variables? We can't visualize them!
- Real-world production, logistics, and scheduling problems often involve many variables and constraints.
- We need a systematic, algebraic method to find the optimal solution.
- The **Simplex Method** (developed by George Dantzig) is the classic and widely used algorithm for solving Linear Programming problems.

The Core Idea of the Simplex Method

- Key insight for our nonnegative-variable LP: if a finite optimum exists, at least one optimal solution is a **vertex**. Other optimal points may lie along an edge.
- The Simplex method starts at an initial corner point (usually the origin if feasible).
- It then iteratively moves from the current corner point to an *adjacent* corner point.
- A pivot preserves feasibility and does not decrease the objective. Degenerate pivots can leave the objective and geometric point unchanged.
- The algorithm stops when it reaches a corner point where no adjacent corner point offers a better objective function value. This point is the optimal solution.
- Practical solvers handle initialization, degeneracy and numerical tolerances. Worst-case running time can still be exponential.

Problem: Maximize $Z = 3x_1 + 5x_2$, where Z is weekly profit in thousands of dollars.

- Graphical solution found the optimum at $(x_1, x_2) = (2, 6)$ with $Z = 36$.
- The Simplex method finds this optimum algebraically.
- **Core Idea:** It explores the *corner points* (vertices) of the feasible region systematically. These points are called Basic Feasible Solutions (BFS).

Corner Points (BFS) & Objective Values

The Simplex method evaluates the objective function $Z = 3x_1 + 5x_2$ at the corner points:

- BFS (0, 0) $\rightarrow Z = 0$
- BFS (4, 0) $\rightarrow Z = 12$
- BFS (0, 6) $\rightarrow Z = 30$
- BFS (4, 3) $\rightarrow Z = 27$
- **BFS (2, 6) $\rightarrow Z = 36$**

Simplex doesn't necessarily check all points; it moves between adjacent points.

Simplex Path & Optimality

The Simplex algorithm iteratively improves the solution:

- **Start:** Typically begins at a BFS, e.g., (0,0) with Z=0.
- **Move:** A pivot changes the basis. Here each pivot reaches an adjacent vertex and increases Z.
  - Example Path: (0,0) [Z=0] $\rightarrow$ (0,6) [Z=30] $\rightarrow$ (2,6) [Z=36]
- **Stop (Optimality):** The algorithm terminates at a BFS when no *adjacent* BFS yields a higher objective function value.
  - At (2,6), adjacent BFS (4,3) [Z=27] and (0,6) [Z=30] are not better.
- **Result:** (2,6) is identified as the optimal solution.

Essence: An efficient search along the edges of the feasible region.

Formal Simplex: Step 1 - Standard Form

To apply the Simplex algorithm, we first convert the LP into **Standard Form**:

- Objective is maximization.
- All constraints are equations with non-negative Right Hand Sides (RHS).
- All variables are non-negative.

We introduce **slack variables** ($s_1, s_2, s_3 \geq 0$) for $\leq$ constraints:

Formal Simplex: Wyndor in Standard Form

Original Wyndor Glass:

$$\text{Max } Z = 3x_1 + 5x_2$$

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \\ x_1, x_2 &\geq 0 \end{aligned}$$

Standard Form:

$$\begin{array}{lrrrrrrl} \text{Max } Z & & & & & & & \\ \text{s.t.} & x_1 & & & +s_1 & & & = 4 \\ & & 2x_2 & & & +s_2 & & = 12 \\ & 3x_1 & +2x_2 & & & & +s_3 & = 18 \\ & x_1, & x_2, & s_1, & s_2, & s_3 & \geq 0 & \end{array}$$

Also, rewrite the objective as: $Z - 3x_1 - 5x_2 = 0$

Formal Simplex: Step 2 - Initial Tableau

The Standard Form equations are organized into a **Simplex Tableau**.

- Each row represents a constraint or the objective function.
- Columns for each variable (decision, slack) and the RHS.
- The 'Basic Variable' column shows the variable currently forming the solution for that row. Initially, these are the slack variables.

Initial Wyndor Glass Tableau:

Basic Var.	**Z**	x_1	x_2	s_1	s_2	s_3	**RHS**
Z	1	-3	-5	0	0	0	0
s_1	0	1	0	1	0	0	4
s_2	0	0	2	0	1	0	12
s_3	0	3	2	0	0	1	18

Initial Basic Feasible Solution (BFS): $x_1 = 0, x_2 = 0, s_1 = 4, s_2 = 12, s_3 = 18$.
Objective Value $Z = 0$.

Simplex Iteration: Optimality & Entering Variable

1. **Optimality Check:**
   - Examine the coefficients in the Z-row (top row).
   - If all coefficients for non-basic variables (x_1, x_2 here) are ≥ 0, the current solution is optimal.
   - *Here: Coefficients for x_1 (-3) and x_2 (-5) are negative. Solution is NOT optimal.*
2. **Select Entering Variable (Pivot Column):**
   - One pivot rule chooses the non-basic variable with the *most negative* Z-row coefficient. This gives the greatest gain per unit, but the ratio test limits the step.
   - *Here: -5 is the most negative.* $\mathbf{x_2}$ *is the entering variable.*
   - The column corresponding to x_2 is the **Pivot Column**.

Basic Var.	**Z**	x_1	$\mathbf{x_2}$	s_1	s_2	s_3	**RHS**
Z	1	-3	-5	0	0	0	0
s_1	0	1	0	1	0	0	4
s_2	0	0	2	0	1	0	12
s_3	0	3	2	0	0	1	18

3. Select Leaving Variable (Pivot Row) - Min Ratio Test:

- For each constraint row where the pivot column entry is > 0, compute the ratio: RHS / Pivot Column Entry.
- *Row s_1: No ratio (entry is 0)*
- *Row s_2: Ratio* $= 12/2 = 6$
- *Row s_3: Ratio* $= 18/2 = 9$
- Choose the row with the *minimum non-negative ratio*.
- *Here: Minimum ratio is 6, corresponding to row s_2. s_2 is the leaving variable.*
- The row s_2 is the **Pivot Row**.

4. Pivot Element & Pivoting:

- The element at the intersection of the Pivot Column (x_2) and Pivot Row (s_2) is the **Pivot Element** (here, it's 2).
- Perform row operations to make the pivot element 1 and all other entries in the pivot column 0. This updates the tableau to reflect the new BFS.

Basic Var.	**Z**	x_1	$\mathbf{x_2}$	s_1	s_2	s_3	**RHS**	**Ratio**
Z	1	-3	-5	0	0	0	0	
s_1	0	1	0	1	0	0	4	—
s_2	0	0	2	0	1	0	12	$12/2 = \mathbf{6}$ **(Min)**
s_3	0	3	2	0	0	1	18	$18/2 = 9$

Performing the Pivot Operations (Iteration 1)

Goal: Use row operations to make the pivot element [2] (s_2 row, x_2 col) become 1, and other entries in the x_2 column become 0.

Operations (Using R_{s2} as the pivot row):

- **New** R_{s2}**'** $=$ R_{s2} **/ 2** (Makes pivot element 1)
- **New** R_Z**'** $=$ R_Z **+ 5 * (New** R_{s2}**')** (Makes -5 in x_2 col zero)
- **New** R_{s1}**'** $=$ R_{s1} **- 0 * (New** R_{s2}**')** (R_{s1} unchanged as entry is 0)
- **New** R_{s3}**'** $=$ R_{s3} **- 2 * (New** R_{s2}**')** (Makes 2 in x_2 col zero)

Resulting Tableau: (x_2 replaces s_2 as basic variable)

Basic Var.	**Z**	x_1	x_2	s_1	s_2	s_3	**RHS**
Z	1	-3	0	0	5/2	0	30
s_1	0	1	0	1	0	0	4
x_2	0	0	1	0	1/2	0	6
s_3	0	3	0	0	-1	1	6

New BFS: $(0, 6)$, $Z = 30$. *Set nonbasic* $x_1 = s_2 = 0$.

Simplex Iteration 2: Optimality & Entering Variable

We start with the tableau from the end of Iteration 1:

Basic Var.	**Z**	x_1	x_2	s_1	s_2	s_3	**RHS**
Z	1	-3	0	0	5/2	0	30
s_1	0	1	0	1	0	0	4
x_2	0	0	1	0	1/2	0	6
s_3	0	3	0	0	-1	1	6

1. Optimality Check:

- Examine the Z-row coefficients for non-basic variables (x_1, s_2).
- The coefficient for x_1 is -3 (negative).
- *Solution is NOT optimal.*

2. Select Entering Variable (Pivot Column):

- Choose the variable with the most negative coefficient in the Z-row.
- *Here: -3 is the only negative.* $\mathbf{x_1}$ *is the entering variable.*
- The column corresponding to x_1 is the **Pivot Column**.

Simplex Iteration 2: Leaving Variable & Pivot

3. Select Leaving Variable (Pivot Row) - Min Ratio Test:

- For each row where the pivot column (x_1) entry is positive: Ratio = RHS / Pivot Column Entry.
- *Row s_1: Ratio = 4 / 1 = 4*
- *Row x_2: No ratio (entry is 0)*
- *Row s_3: Ratio = 6 / 3 = 2*
- Choose the row with the *minimum non-negative ratio*.
- *Here: Minimum ratio is 2, corresponding to row* **s_3**. *s_3 is the leaving variable.*

4. Pivot Element:

- The element at the intersection of the Pivot Column (x_1) and Pivot Row (s_3) is the **Pivot Element** (here, it's 3).

Basic Var.	Z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
Z	1	**-3**	0	0	5/2	0	30	
s_1	0	1	0	1	0	0	4	$4/1 = 4$
x_2	0	0	1	0	1/2	0	6	—
s_3	0	**[3]**	0	0	-1	1	6	$6/3 =$ **2 (Min)**

Simplex Iteration 2: Pivot Operations

Goal: Make pivot element [3] become 1, and other entries in pivot column (x_1) become 0.

Operations (Using R_{s3} as the pivot row):

- New R_{s3}' = R_{s3} / 3
- New R_Z' = R_Z + 3 * (New R_{s3}')
- New R_{s1}' = R_{s1} - 1 * (New R_{s3}')
- New R_{x2}' = R_{x2} - 0 * (New R_{s3}') (No change)

Resulting Tableau: (x_1 replaces s_3 as basic variable)

Basic Var.	**Z**	x_1	x_2	s_1	s_2	s_3	**RHS**
Z	1	0	0	0	3/2	1	36
s_1	0	0	0	1	1/3	-1/3	2
x_2	0	0	1	0	1/2	0	6
x_1	0	1	0	0	-1/3	1/3	2

Optimal Solution Reached

Let's examine the Z-row of the latest tableau:

Basic Var.	Z	x_1	x_2	s_1	s_2	s_3	RHS
Z	1	**0**	**0**	**0**	**3/2**	**1**	**36**
...	...	...	...	...	...	...	...

Optimality Check:

- All coefficients in the Z-row for non-basic variables (s_2, s_3) are non-negative (3/2 and 1).
- **The current solution is optimal!**

Optimal Solution:

- Read values from the 'Basic Var.' column and 'RHS':
- $x_1 = 2$ (from x_1 row)
- $x_2 = 6$ (from x_2 row)
- $s_1 = 2$ (from s_1 row, slack in Plant 1 capacity)
- Non-basic variables are zero: $s_2 = 0, s_3 = 0$ (Plants 2 & 3 used fully)
- Maximum Objective Value: $Z = 36$ (from RHS in Z-row)

This matches the graphical solution result.

Simplex Process Summary

The Iterative Process:

1. Start with a feasible basis. Wyndor uses slack variables. General LPs may need a separate feasibility phase.
2. **Repeat** the following steps:
   - Check for Optimality (Z-row coefficients ≥ 0?). If yes, STOP.
   - Select Entering Variable (most negative Z-row coefficient).
   - Select Leaving Variable (minimum ratio test). No positive pivot entry with an improving column means unboundedness.
   - Pivot to update the tableau and find the new BFS.

For Wyndor Glass:

- **Initial Tableau:** BFS=(0,0), Z=0. Pivot on x_2 entering, s_2 leaving.
- **Tableau 1:** BFS=(0,6), Z=30. (Not optimal, Z-row coeff for x_1 is negative). Pivot on x_1 entering, s_3 leaving.
- **Tableau 2:** BFS=(2,6), Z=36. (Z-row coefficients are all non-negative). **OPTIMAL!**

Capacity Has a Local Value

- One extra Plant 1 hour leaves the optimum unchanged: two hours are already idle.
- With Plant 3 capacity 19, the plan becomes $(7/3, 6)$ and $Z = 37$.
- Initially, one Plant 3 hour adds \$1000 per week. This is its **shadow price**.
- The rate holds for Plant 3 capacity from 12 to 24 hours. Above 24, Plants 1 and 2 prevent further gain.

Decision: five extra Plant 3 hours for \$4000/week add \$5000 contribution, so the net gain is \$1000/week under these assumptions.

The Wyndor model in PuLP

```
model = pulp.LpProblem("Wyndor", pulp.LpMaximize)
doors = pulp.LpVariable("door_batches", lowBound=0)
windows = pulp.LpVariable("window_batches", lowBound=0)
model += 3*doors + 5*windows, "weekly_contribution"
model += doors <= 4, "plant_1_hours"
model += 2*windows <= 12, "plant_2_hours"
model += 3*doors + 2*windows <= 18, "plant_3_hours"
model.solve(pulp.PULP_CBC_CMD(msg=False))
```

Check `pulp.LpStatus[model.status]` before reading values. For this continuous LP, the named constraints expose `.slack` and `.pi` (shadow price). The objective directly maximizes contribution. The companion imports `pulp` and provides the complete experiment.

Lecture Companion and Lab Connection

- `notebooks/lecture_1_linear_programming.ipynb`: a 15-minute Wyndor demonstration in PuLP 3.x/CBC.
- Predict which resource has value, move capacity and profit controls, then interpret the new plan.
- Compare a local shadow-price prediction with a full capacity sweep.
- Lab 1 uses a separate robot-kit mix in PuLP. Lab 3 develops transport and network-flow models.

Exit question: Why can a solver return an optimal solution to a model that gives an unusable production plan?

Key Takeaways

- Classic example of a product-mix problem in linear programming
- Demonstrates allocation of limited resources to maximize profit
- Shows how production control problems can be translated into mathematical models
- Shows value of OR methods in practical decision-making
- Highlights importance of clear problem definition and data gathering
- Simplex method provides an algorithmic way to solve such problems, even with many variables.

Assumptions of Linear Programming

Linear Programming models rely on several key assumptions:

- **Proportionality:** Doubling a variable doubles its contribution to profit, cost, and resource usage.
- **Additivity:** Total profit and total resource usage are sums of individual contributions; activities do not interact.
- **Divisibility:** Decision variables are continuous, so fractional values such as 2.5 batches are allowed.
- **Certainty:** Model parameters such as profits, coefficients, and capacities are known and fixed.

When these assumptions do not hold, extensions such as integer, nonlinear, or stochastic programming are used.

Typical LP Application: Blending Problem

Blending problems are another common application of Linear Programming.

- **Goal:** Determine the least expensive combination (blend) of raw materials or ingredients to produce a final product that meets specific quality standards or compositional requirements.
- **Common Examples:**
  - Mixing ingredients for animal feed or fertilizers.
  - Blending different types of crude oils to produce gasoline grades.
  - Combining ingredients for food products (like the Whiskas example).
- **Key Elements:** Costs of ingredients, nutritional/chemical composition of ingredients, required specifications for the final product.

Example: A 100 g Least-Cost Blend

Teaching example: choose chicken and beef to make one 100 g can.

Per gram of ingredient	Chicken	Beef
Cost (dollars/g)	0.013	0.008
Protein (g/g)	0.100	0.200
Fat (g/g)	0.080	0.100
Fibre (g/g)	0.001	0.005
Salt (g/g)	0.002	0.005

Per-can requirements: protein ≥ 8 g, fat ≥ 6 g, fibre ≤ 2 g, salt ≤ 0.4 g. **Predict:** Beef is cheaper. Which requirement prevents using only beef?

Whiskas Example (Simplified) - LP Formulation

Decision Variables:

- Let x_1 = grams of chicken in a 100g can.
- Let x_2 = grams of beef in a 100g can.

Objective Function (Minimize Cost):

$$\min Z = 0.013x_1 + 0.008x_2$$

Constraints:

- **Total mass (g):** $x_1 + x_2 = 100$
- **Protein Req:** $0.100x_1 + 0.200x_2 \geq 8.0$
- **Fat Req:** $0.080x_1 + 0.100x_2 \geq 6.0$
- **Fibre Limit:** $0.001x_1 + 0.005x_2 \leq 2.0$
- **Salt Limit:** $0.002x_1 + 0.005x_2 \leq 0.4$
- **Non-negativity:** $x_1 \geq 0, x_2 \geq 0$

Solution: salt forces $x_1 \geq 100/3$ g. Since chicken costs more, choose $x_1 = 100/3$, $x_2 = 200/3$. Cost = \$0.9667 per can.

Typical LP Application: Transportation Problem

One of the earliest and most common applications of Linear Programming.

- **Goal:** Determine the optimal plan for shipping goods from multiple **sources** (e.g., factories, warehouses) to multiple **destinations** (e.g., customers, stores) to minimize total transportation cost.
- **Given Information:**
  - Supply capacity at each source.
  - Demand requirement at each destination.
  - Unit cost of transportation from each source to each destination.
- **Decision:** How much to ship from each source to each destination?
- We will use a 'Beer Distribution' problem as an example.

Example: Beer Distribution - Problem Setup

A brewery needs to ship beer from 2 warehouses (A, B) to 5 bars (1-5) at minimum cost.

Data:

- **Supply:** Warehouse A: 1000 crates, Warehouse B: 4000 crates.
- **Demand:** Bar 1: 500, Bar 2: 900, Bar 3: 1800, Bar 4: 200, Bar 5: 700 crates.
- **Shipping Costs** ($/crate). Supply = 5000, demand = 4100 crates: 900 may remain in stock.

	Bar 1	**Bar 2**	**Bar 3**	**Bar 4**	**Bar 5**
Warehouse A	2	4	5	2	1
Warehouse B	3	1	3	2	3

Check before solving: total stock exceeds total orders by 900 crates. Should all stock have to ship?

Beer Distribution: A Feasible Transportation Model

Let $x_{wb} \geq 0$ be crates from warehouse w to bar b. Costs c_{wb} come from the preceding table.

$$\min Z = \sum_{w \in \{A,B\}} \sum_{b=1}^{5} c_{wb} x_{wb}.$$

$$\sum_{b=1}^{5} x_{Ab} \leq 1000, \quad \sum_{b=1}^{5} x_{Bb} \leq 4000,$$

$$x_{Ab} + x_{Bb} = d_b \qquad (b = 1, \ldots, 5).$$

- Exact orders require 4100 crates. Supply limits allow 900 to remain in stock.
- Replacing both supply limits by equalities would force $5000 = 4100$ and make the model infeasible.
- With integer supplies and demands, this basic transportation LP has an integer optimal vertex.

The Transportation Plan and Its Cost

Crates	Bar 1	Bar 2	Bar 3	Bar 4	Bar 5
Warehouse A	300	0	0	0	700
Warehouse B	200	900	1800	200	0

- A ships 1000, B ships 3100, and B retains 900 crates. Every order is met.
- Total shipping cost is \$8600.
- Compared with shipping everything from B, an A crate saves \$2 for Bar 5 and \$1 for Bar 1. The best use of A is therefore Bar 5 first, then Bar 1.

Predict: Would an extra 100 crates of A capacity help? Which route changes?

Transportation Problem - Balancing

Often, Total Supply might not equal Total Demand.

- **Balanced case:** If total supply equals total demand, transportation constraints are usually modeled as equalities.
- **If Supply $>$ Demand:** Extra supply can be modeled by shipping to a *dummy demand node* (representing storage, cost often 0).
- **If Demand $>$ Supply:** The problem is infeasible unless additional supply is modeled. This can be done using a *dummy supply node* (representing buying-in, substitution, or unmet demand penalty cost).

For our data: allow unused supply, or add a storage node demanding 900 crates. Its cost must reflect the actual holding cost.
Balancing ensures all supply/demand is accounted for in the model.